asio官方案例存在的隐患

Posted on 2023-04-10 11:17:19 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2ODYV1A2xbhTjWr0FJ1ZS22ijZO)

简介

前文已经介绍了异步操作的api，今天写一个简单的异步echo服务器，以应答为主

Session类

Session类主要是处理客户端消息收发的会话类，为了简单起见，我们不考虑粘包问题，也不考虑支持手动调用发送的接口，只以应答的方式发送和接收固定长度(1024字节长度)的数据。

<!--more-->

```
1. class Session
2. {
3. public:
4.     Session(boost::asio::io_context& ioc):_socket(ioc){
5.     }
6.     tcp::socket& Socket() {
7.         return _socket;
8.     }
9.
10.    void Start();
11. private:
12.    void handle_read(const boost::system::error_code & error, size_t bytes_transfered);
13.    void handle_write(const boost::system::error_code& error);
14.    tcp::socket _socket;
15.    enum {max_length = 1024};
16.    char _data[max_length];
17. };
```

1 _data用来接收客户端传递的数据

2 _socket为单独处理客户端读写的socket。

3 handle_read和handle_write分别为读回调函数和写回调函数。

接下来我们实现Session类

```
void Session::Start(){
    memset(_data, 0, max_length);
    _socket.async_read_some(boost::asio::buffer(_data, max_length),
        std::bind(&Session::handle_read, this, placeholders::_1,
            placeholders::_2)
    );
}
```

在Start方法中我们调用异步读操作，监听对端发送的消息。当对端发送数据后，触发handle_read函数

```
void Session::handle_read(const boost::system::error_code& error, size_t bytes_transfered) {

    if (!error) {
        cout << "server receive data is " << _data << endl;
        boost::asio::async_write(_socket, boost::asio::buffer(_data, bytes_transfered),
            std::bind(&Session::handle_write, this, placeholders::_1));
    }
    else {
        delete this;
    }
}
```

handle_read函数内将收到的数据发送给对端，当发送完成后触发handle_write回调函数。

```
void Session::handle_write(const boost::system::error_code& error) {
    if (!error) {
        memset(_data, 0, max_length);
        _socket.async_read_some(boost::asio::buffer(_data, max_length), std::bind(&Session::handle_read,
            this, placeholders::_1, placeholders::_2));
    }
    else {
        delete this;
    }
}
```

handle_write函数内又一次监听了读事件，如果对端有数据发送过来则触发handle_read，我们再将收到的数据发回去。从而达到应答式服务的效果。

Server类

Server类为服务器接收连接的管理类

```
1. class Server {
2. public:
3.     Server(boost::asio::io_context& ioc, short port);
4. private:
5.     void start_accept();
6.     void handle_accept(Session* new_session, const boost::system::error_code& error);
7.     boost::asio::io_context& _ioc;
8.     tcp::acceptor _acceptor;
9. };
```

start_accept将要接收连接的acceptor绑定到服务上，其内部就是将accpeptor对应的socket描述符绑定到epoll或iocp模型上，实现事件驱动。
handle_accept为新连接到来后触发的回调函数。
下面是具体实现

```
Server::Server(boost::asio::io_context& ioc, short port) :_ioc(ioc),
_acceptor(ioc, tcp::endpoint(tcp::v4(), port)) {
    start_accept();
}

void Server::start_accept() {
    Session* new_session = new Session(_ioc);
    _acceptor.async_accept(new_session->Socket(),
        std::bind(&Server::handle_accept, this, new_session, placeholders::_1));
}
void Server::handle_accept(Session* new_session, const boost::system::error_code& error) {
    if (!error) {
        new_session->Start();
    }
    else {
        delete new_session;
    }

    start_accept();
}
```

客户端

客户端的设计用之前的同步模式即可，客户端不需要异步的方式，因为客户端并不是以并发为主，当然写成异步收发更好一些。

```cpp
#include <iostream>
#include <boost/asio.hpp>
using namespace std;
using namespace boost::asio::ip;
const int MAX_LENGTH = 1024;
int main()
{
    try {
        //创建上下文服务
        boost::asio::io_context   ioc;
        //构造endpoint
        tcp::endpoint  remote_ep(address::from_string("127.0.0.1"), 10086);
        tcp::socket  sock(ioc);
        boost::system::error_code   error = boost::asio::error::host_not_found; ;
        sock.connect(remote_ep, error);
        if (error) {
            cout << "connect failed, code is " << error.value() << " error msg is " << error.message
    ();
            return 0;
        }

        std::cout << "Enter message: ";
        char request[MAX_LENGTH];
        std::cin.getline(request, MAX_LENGTH);
        size_t request_length = strlen(request);
        boost::asio::write(sock, boost::asio::buffer(request, request_length));

        char reply[MAX_LENGTH];
        size_t reply_length = boost::asio::read(sock,
            boost::asio::buffer(reply, request_length));
        std::cout << "Reply is: ";
        std::cout.write(reply, reply_length);
        std::cout << "\n";
    }
    catch (std::exception& e) {
        std::cerr << "Exception: " << e.what() << endl;
    }
    return 0;
}
```

运行服务器之后再运行客户端，输入字符串后，就可以收到服务器应答的字符串了。

隐患

该demo示例为仿照asio官网编写的，其中存在隐患，就是当服务器即将发送数据前(调用async_write前)，此刻客户端中断，服务器此时调用async_write会触发发送回调函数，判断ec为非0进而执行delete this逻辑回收session。但要注意的是客户端关闭后，在tcp层面会触发读就绪事件，服务器会触发读事件回调函数。在读事件回调函数中判断错误码ec为非0，进而再次执行delete操作，从而造成二次析构，这是极度危险的。

总结

本文介绍了异步的应答服务器设计，但是这种服务器并不会在实际生产中使用，主要有两个原因:

1　因为该服务器的发送和接收以应答的方式交互，而并不能做到应用层想随意发送的目的，也就是未做到完全的收发分离(全双工逻辑)。

2　该服务器未处理粘包，序列化，以及逻辑和收发线程解耦等问题。

3　该服务器存在二次析构的风险。

这些问题我们会在接下来的文章中不断完善

源码链接

https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3732) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(555) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)